

АННОТАЦИЯ

Варданян В.А. Разработка веб-приложения «Умный диск» (облачное хранилище) с RAG-архитектурой и чат-доступом к содержимому: выпускная квалификационная работа бакалавра / Варданян В.А.; РТУ МИРЭА, ИПТИП, кафедра индустриального программирования; руководитель Пальчевский Е.В. — Москва, 2026.

Объём работы: 58 страниц основного текста, 15 рисунков, 6 таблиц, 1 приложение, 35 использованных источников.

Ключевые слова: облачное хранилище, RAG, Retrieval-Augmented Generation, семантический поиск, векторные представления, эмбединги, bge-m3, bge-reranker, Qwen3, llama.cpp, FastAPI, React, PostgreSQL, pgvector, чат-интерфейс, JWT-аутентификация.

Разработано веб-приложение «Умный диск» — интеллектуальное облачное хранилище документов на основе архитектуры Retrieval-Augmented Generation (RAG). Система обеспечивает загрузку и хранение файлов форматов PDF, DOCX, TXT и Markdown, автоматическое индексирование с генерацией эмбедингов через локальную модель bge-m3, семантический поиск с переранжированием через bge-reranker-base и чат-интерфейс на основе локальной языковой модели Qwen3-4B. Бэкенд — FastAPI + async SQLAlchemy + JWT, фронтенд — React + Vite, векторное хранилище — PostgreSQL + pgvector. По результатам тестирования на датасете из 50 вопросов семантический поиск обеспечил $\text{Recall}@5 = 0,84$, чат-модуль — 84% корректных ответов.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

RAG — Retrieval-Augmented Generation (генерация с дополненным поиском)

LLM — Large Language Model (большая языковая модель)

API — Application Programming Interface (программный интерфейс приложения)

REST — Representational State Transfer

JWT — JSON Web Token

NLP — Natural Language Processing (обработка естественного языка)

SPA — Single Page Application (одностраничное веб-приложение)

ORM — Object-Relational Mapping (объектно-реляционное отображение)

ANN — Approximate Nearest Neighbor (приближённый поиск ближайших соседей)

UI — User Interface (пользовательский интерфейс)

БД — база данных

ВКР — выпускная квалификационная работа

ПО — программное обеспечение

СУБД — система управления базами данных

MVP — Minimum Viable Product (минимально жизнеспособный продукт)

СОДЕРЖАНИЕ

АННОТАЦИЯ	1
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	2
СОДЕРЖАНИЕ.....	3
ВВЕДЕНИЕ	4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ	7
1.1 Облачные системы хранения данных: анализ и ограничения	7
1.2 Технологии семантического поиска и векторных представлений	8
1.3 Архитектура Retrieval-Augmented Generation	9
1.4 Сравнительный анализ и обоснование выбора векторного хранилища	11
1.5 Сравнительный анализ и обоснование выбора модели эмбеддингов	12
1.6 Сравнительный анализ и обоснование выбора языковой модели	14
1.7 Сравнительный анализ и обоснование выбора серверного фреймворка.....	15
1.8 Выводы по первой главе.....	16
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ «УМНЫЙ ДИСК».	17
2.1 Требования к веб-приложению	17
2.2 Общая архитектура системы	18
2.3 Стек технологий и обоснование выбора	20
2.4 Модель данных и схема базы данных	21
2.5 Конвейер индексирования документов	23
2.6 Конвейер обработки запросов с переранжированием	25
2.7 Реализация бэкенда.....	27
2.8 Реализация фронтенда	28
2.9 Выводы по второй главе	29
3 ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ СИСТЕМЫ	30
3.1 Методология тестирования	30
3.2 Результаты тестирования семантического поиска	33
3.3 Результаты тестирования чат-модуля	34
3.4 Тестирование производительности	35
3.5 Функциональное тестирование	36
3.6 Выводы по третьей главе	38
ЗАКЛЮЧЕНИЕ	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	41
ПРИЛОЖЕНИЕ А	45

ВВЕДЕНИЕ

Корпоративные пользователи ежедневно работают с десятками и сотнями электронных документов — договорами, техническими спецификациями, отчётами. Популярные облачные хранилища (Google Drive, Яндекс.Диск, Dropbox) решают задачу хранения, но поиск в них работает по ключевым словам или метаданным файла, не учитывая смысл содержимого [1]. Найти договор аренды, который в тексте называется «соглашением о найме помещения», стандартными средствами нельзя.

Проблема усугубляется, когда нужно не просто найти файл, а извлечь конкретный факт из нескольких документов сразу. Пользователь вынужден открывать каждый файл вручную, читать и сопоставлять фрагменты — трудоёмкий процесс, который не масштабируется при росте коллекции.

Архитектура Retrieval-Augmented Generation (RAG), предложенная Meta AI Research в 2020 году [2], даёт практичный ответ на эту проблему: документы преобразуются в векторные представления, при запросе система находит семантически близкие фрагменты и передаёт их языковой модели как контекст. Модель отвечает строго по документам, а не по параметрической памяти, что резко снижает риск «галлюцинаций» [12].

Дополнительный аргумент в пользу локальной реализации — конфиденциальность: корпоративные документы нельзя отправлять во внешние API. Появление компактных моделей класса Qwen3-4B и bge-m3, работающих на обычном CPU, делает полностью локальную RAG-систему реально работающей на рабочей станции без GPU.

Объект исследования является система облачного хранения документов и интеллектуального поиска информации.

Предмет исследования является метод семантического поиска и обработка текстовых документов на основе архитектуры Retrieval-Augmented Generation.

Цель работы: разработать веб-приложение «Умный диск» — MVP интеллектуального облачного хранилища файлов с архитектурой RAG, обеспечивающее хранение документов, семантический поиск по их содержимому с переранжированием и чат-интерфейс на основе локальной языковой модели.

Для достижения цели поставлены следующие задачи:

- 1) провести анализ предметной области облачных систем хранения данных, технологий интеллектуального поиска и RAG-архитектуры, выполнить обзор существующих инструментов и решений;
- 2) сформировать функциональные и нефункциональные требования к разрабатываемому веб-приложению;
- 3) разработать архитектуру системы и обосновать выбор стека технологий: FastAPI, React + Vite, PostgreSQL + pgvector, bge-m3, bge-reranker-base, Qwen3-4B через llama.cpp;
- 4) реализовать конвейер индексирования документов: извлечение текста, чанкинг, генерация эмбедингов bge-m3, сохранение в pgvector;
- 5) реализовать чат-модуль с двухэтапным поиском (bge-m3 + bge-reranker-base) и генерацией ответов через Qwen3-4B;
- 6) провести тестирование системы и оценить качество семантического поиска, переранжирования и генерации ответов.

Практическая значимость работы состоит в создании функционирующего MVP, которое не зависит ни от платных API, ни от GPU. В ходе разработки три технических решения потребовали наибольшего

внимания: выбор размера чанка (слишком малый размер ухудшал контекстность, слишком большой — перегружал reranker), подбор параметра lists IVFFlat-индекса под объём коллекции, а также стратегия фильтрации по user_id прямо в векторном запросе, а не постфактум. Работа состоит из введения, трёх глав, заключения, списка использованных источников и приложения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1 Облачные системы хранения данных: анализ и ограничения

Облачное хранение данных — модель, при которой файлы размещаются на удалённых серверах и доступны пользователям по сети [3]. Сегодня облачные хранилища стали стандартной частью корпоративной и персональной инфраструктуры.

Ведущие платформы — Google Drive, Microsoft OneDrive, Dropbox, Box, Яндекс.Диск — предоставляют примерно одинаковый набор: хранение файлов любых форматов, иерархию папок, совместный доступ, синхронизацию между устройствами и поиск по названию и метаданным [4].

Общее ограничение всех перечисленных сервисов — поиск по ключевым словам с точным или нечётким совпадением строк [5]. Семантическая близость понятий не учитывается: запрос «договор аренды» не найдёт «соглашение о найме помещения». Возможность задать вопрос на естественном языке и получить синтезированный ответ ни в одном из них не предусмотрена.

Таблица 1.1 фиксирует это сравнение по функциональным характеристикам.

Таблица 1.1 – Сравнение функциональных возможностей облачных хранилищ

Функция	Google Drive	Яндекс.Диск	Dropbox	Notion AI	Умный диск
Хранение файлов	Да	Да	Да	Да	Да
Поиск по названию	Да	Да	Да	Да	Да
Полнотекстовый поиск	Частично	Нет	Нет	Да	Да
Семантический поиск	Нет	Нет	Нет	Частично	Да
Reranking	Нет	Нет	Нет	Нет	Да
Чат-интерфейс	Нет	Нет	Нет	Да	Да
Локальные модели	Нет	Нет	Нет	Нет	Да
Открытый исходный код	Нет	Нет	Нет	Нет	Да

Ни одна из рассмотренных платформ не сочетает семантический поиск, reranking и локальную LLM одновременно — именно это сочетание реализуется в данной работе.

1.2 Технологии семантического поиска и векторных представлений

Семантический поиск строится на идее, что смысл текста можно выразить числовым вектором: семантически близкие фрагменты оказываются геометрически близки в многомерном пространстве [6]. Это позволяет находить документы по смыслу запроса, а не по буквальному совпадению слов.

Современные embedding-модели основаны на архитектуре трансформера [8]; такие модели, как BERT [9] и его производные, кодируют запрос и документ независимо (bi-encoder), что допускает предварительное вычисление и кэширование эмбеддингов документов [7].

Bi-encoder быстр, но уступает в точности: он оценивает запрос и документ раздельно. Cross-encoder (reranker) рассматривает пару совместно и

даёт более точный скор релевантности ценой большего времени [10]. В проекте применяется двухэтапная схема: bi-encoder bge-m3 отбирает top-20 кандидатов, bge-reranker-base из них выбирает top-5 для контекста.

Размер чанка — важный практический параметр: слишком малый теряет контекст, слишком большой снижает точность retrieval. Литература рекомендует 256–512 токенов с перекрытием 10–20% [11]. В проекте выбрано символьное разбиение (512 символов, перекрытие 50) — это независимо от токенизатора и даёт предсказуемый размер фрагмента на любом языке.

1.3 Архитектура Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) — архитектурный паттерн, сочетающий извлечение релевантных фрагментов из базы знаний (retrieval) с генерацией связного ответа на их основе с помощью языковой модели (generation) [2]. Концепция была формализована Льюисом и соавторами из Meta AI Research в 2020 году и с тех пор получила широкое применение.

В отличие от чистых генеративных моделей, RAG-системы динамически обращаются к внешней базе знаний при каждом запросе. Это позволяет преодолеть ключевые ограничения LLM: устаревание знаний после даты отсечки обучения и склонность к «галлюцинациям» — генерации правдоподобно звучащей, но фактически неверной информации [12]. Ответ формируется исключительно на основе предоставленного контекста, что обеспечивает верифицируемость и прозрачность источников.

Стандартный конвейер RAG включает следующие этапы:

- 1) индексирование — разбиение документов на фрагменты, генерация эмбедингов, сохранение в векторное хранилище;
- 2) получение запроса пользователя на естественном языке;
- 3) retrieval — преобразование запроса в эмбединг, ANN-поиск top-k ближайших фрагментов;
- 4) reranking — переранжирование результатов cross-encoder-моделью;
- 5) augmentation — объединение лучших фрагментов с запросом в промпт для языковой модели;
- 6) generation — языковая модель генерирует ответ на основе предоставленного контекста.

Различают Naive RAG (прямой конвейер без reranking), Advanced RAG (с переранжированием и улучшенной стратегией retrieval) и Modular RAG с гибкой компоновкой блоков [13]. Реализованная система соответствует классу Advanced RAG.

1.4 Сравнительный анализ и обоснование выбора векторного хранилища

Выбор инструмента для хранения и поиска векторных представлений является одним из ключевых архитектурных решений при построении RAG-системы. Основными кандидатами являются специализированные векторные базы данных (Pinecone [15], Qdrant [16], Weaviate [17], Chroma [33]) и расширение pgvector для PostgreSQL [14]. В таблице 1.2 приведено их сравнение по критериям, существенным для разрабатываемой системы.

Таблица 1.2 – Сравнение инструментов векторного поиска

Критерий (вес)	pgvector	Qdrant	Pinecone	Chroma
Единая инфраструктура (0,25)	3	1	1	2
Производительность при $n \leq 10^6$ (0,20)	2	3	3	2
Локальность / без внешних API (0,25)	3	2	1	3
Простота развёртывания (0,15)	3	2	3	2
Фильтрация по метаданным (0,15)	3	3	3	2
Итоговый балл S	2,80	2,10	2,00	2,35

Критерии и их веса определены из требований проекта (локальность, единая инфраструктура). Для каждого решения введена балльная оценка 1–3, итоговый взвешенный балл S:

$$S = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + w_4 \cdot x_4 + w_5 \cdot x_5,$$

где x_1 — оценка по критерию «единая инфраструктура» (вес $w_1 = 0,25$); x_2 — «производительность при $n \leq 10^6$ векторов» ($w_2 = 0,20$); x_3 — «отсутствие зависимости от внешних сервисов» ($w_3 = 0,25$); x_4 — «сложность

развёртывания» ($w_4 = 0,15$); x_5 — «поддержка фильтрации по метаданным» ($w_5 = 0,15$).

Расчёт для pgvector даёт $S = 2,80$; для Chroma — $2,35$; для Qdrant — $2,10$; для Pinecone — $2,00$.

pgvector побеждает прежде всего по двум критериям с наибольшим весом: единая инфраструктура с PostgreSQL и полная локальность. Косинусное сходство между вектором запроса q и вектором фрагмента d вычисляется стандартным образом:

$$\cos(q, d) = (q \cdot d) / (\|q\| \cdot \|d\|),$$

где $q \cdot d$ — скалярное произведение, $\|\cdot\|$ — евклидова норма. Оператор pgvector `<=>` реализует это расстояние прямо в SQL с фильтрацией по `user_id` в одном запросе [14] — без дополнительного сервиса.

1.5 Сравнительный анализ и обоснование выбора модели эмбеддингов

Качество поиска напрямую зависит от модели эмбеддингов. Для сравнения отобрали четыре локально развёртываемые многоязычные модели: bge-m3 [18], multilingual-e5-large [34], paraphrase-multilingual-mpnet [35] и LaBSE [36]. Основной критерий — $nDCG@10$ по задачам retrieval на бенчмарке MTEB [19], охватывающем десятки языков, включая русский.

Формально задача retrieval: по запросу q и корпусу $D = \{d_1, \dots, d_n\}$ вернуть $R_k \subseteq D$ размером k , близкий к эталонному множеству Gq ; предпочтительна модель с наибольшим $nDCG@10$ на MTEB [19].

Таблица 1.3 – Сравнение моделей эмбедингов для многоязычного retrieval

Модель	Размер-ность	Языков	nDCG@10 (MTEB)	Особенности
bge-m3 (BAAI) [18]	1024	100+	56,9	Dense + Sparse + ColBERT; парный reranker
multilingual-e5-large [34]	1024	94	51,4	Высокое качество; нет парного reranker
paraphrase-multilingual-mpnet [35]	768	50+	39,8	Малый размер; слабее на retrieval
LaBSE [36]	768	109	43,1	Хорошая многоязычность; нет reranker

bge-m3 лидирует по $nDCG@10 = 56,9$ и поддерживает три режима кодирования одновременно: dense (ANN-поиск), sparse (аналог BM25) и ColBERT (многовекторное сопоставление) [18] — это единственная из рассмотренных моделей, пригодная для гибридного поиска. Решающим аргументом оказалось наличие парной модели bge-reranker-base из той же серии: векторные пространства обоих компонентов согласованы, что важно для двухэтапного поиска.

Выходная размерность bge-m3 — 1024, что зафиксировано в типе VECTOR(1024) в схеме БД. Плотный эмбединг текста t :

$$e(t) = \text{normalize}(\text{Encoder_bge-m3}(t)[\text{CLS}]) \in \mathbb{R}^{1024},$$

где [CLS] — вектор специального токена трансформера; L2-нормализация приводит $\|e(t)\| = 1$, после чего косинусное сходство эквивалентно dot-product [18].

1.6 Сравнительный анализ и обоснование выбора языковой модели

Для генерации ответов в RAG-конвейере требуется языковая модель, удовлетворяющая следующим критериям: поддержка русского языка; возможность локального запуска без GPU; размер GGUF-файла не более 5 ГБ (ограничение дискового пространства); качество следования инструкциям в промпте (instruction following). Были рассмотрены четыре кандидата в квантизованном формате GGUF Q4_K_M: Qwen3-4B [22], Llama-3.2-3B [37], Mistral-7B-v0.3 [38] и Phi-3-mini-4k [39].

Таблица 1.4 – Сравнение локальных языковых моделей для RAG-генерации

Модель	Размер GGUF	Скорость (ток./с)	Instruction Score	Русский язык
Qwen3-4B-Q4_K_M [22]	2,7 ГБ	14	8,1	Отличная поддержка
Llama-3.2-3B-Q4_K_M [37]	2,0 ГБ	18	6,8	Слабая поддержка
Mistral-7B-v0.3-Q4_K_M [38]	4,4 ГБ	9	8,5	Хорошая поддержка
Phi-3-mini-4k-Q4_K_M [39]	2,2 ГБ	16	7,4	Удовлетворительная

Instruction Score получен по MT-Bench [40] — стандартному многозадачному бенчмарку для LLM (письмо, рассуждение, код и др.). Качество русскоязычного текста дополнительно оценивалось по BLEU [41] — метрике n-граммного перекрытия с эталоном:

$$\text{BLEU} = \text{BP} \cdot \exp(\sum_n w_n \cdot \log p_n),$$

где p_n — n -граммная точность, $w_n = 1/N$ — равномерные веса (обычно $N = 4$), BP — штраф за краткость:

$$BP = 1, \text{ если } |c| > |r|; BP = \exp(1 - |r|/|c|), \text{ если } |c| \leq |r|,$$

$|c|$ и $|r|$ — длины сгенерированного и эталонного текстов [41]. Qwen3-4B набирает 8,1 по MT-Bench — лучший результат среди моделей весом до 3 ГБ. Mistral-7B даёт 8,5, но занимает 4,4 ГБ и работает в 1,6 раза медленнее. Поддержка русского языка у Qwen3-4B заметно выше, чем у Llama-3.2-3B и Phi-3-mini — это стало решающим критерием для данного проекта.

1.7 Сравнительный анализ и обоснование выбора серверного фреймворка

Для реализации бэкенда рассматривались три Python-фреймворка: FastAPI [23], Flask [42] и Django REST Framework [43]. Выбор фреймворка существенно влияет на производительность обработки HTTP-запросов, удобство разработки и интеграцию с асинхронными операциями (генерация эмбеддингов, обращение к LLM, запросы к БД).

Таблица 1.5 – Сравнение серверных фреймворков

Критерий	FastAPI	Flask	Django REST	Обоснование веса
Нативный async/await	Да	Частично	Частично	Критично для LLM-инференса
Авто-генерация OpenAPI	Да	Нет	Да	Ускоряет разработку и тестирование
Пропускная способность (RPS)	~65 000	~25 000	~22 000	По данным TechEmpower [44]

Критерий	FastAPI	Flask	Django REST	Обоснование веса
Валидация через Pydantic	Встроено	Сторонняя	Сторонняя	Снижает количество ошибок
Избыточность для MVP	Низкая	Низкая	Высокая	Django несёт ORM, auth, admin

FastAPI выигрывает по трём ключевым пунктам. Во-первых, нативный `async/await` на уровне ASGI позволяет не блокировать `event loop` во время долгих операций — генерации эмбедингов и LLM-инференса. Во-вторых, по данным TechEmpower [44], пропускная способность FastAPI (~65 000 RPS) в 2,5 раза выше, чем у Flask. В-третьих, автоматическая генерация OpenAPI-документации ускорила итерации при разработке API. Django REST Framework в проекте избыточен: его ORM, административный интерфейс и встроенная аутентификация не используются, а синхронная природа по умолчанию создала бы ограничения.

1.8 Выводы по первой главе

Ни одно из рассмотренных коммерческих хранилищ не сочетает семантический поиск с `reranking` и локальную LLM. Для построения такой системы выбраны: `pgvector` как векторное хранилище (взвешенный балл $S = 2,80$, лучший среди четырёх кандидатов) — за единую инфраструктуру с PostgreSQL без внешних API; `bge-m3` как `embedding`-модель ($nDCG@10 = 56,9$ на MTEB) с парным `bge-reranker-base`; `Qwen3-4B-Q4_K_M` как языковая модель (Instruction Score 8,1, скорость 14 токен/с, 2,7 ГБ) — лучший баланс качества и ресурсов среди моделей до 3 ГБ; FastAPI — за нативный `async` и пропускную способность ~65 000 RPS.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ «УМНЫЙ ДИСК»

2.1 Требования к веб-приложению

Требования сформированы из анализа ограничений существующих хранилищ и особенностей RAG-архитектуры.

Аутентификация и управление доступом. Пользователи регистрируются по email, пароли хранятся в виде bcrypt-хэша, все защищённые запросы используют JWT-токен. Каждый SQL-запрос к таблицам файлов и чанков содержит фильтр по `user_id` — изоляция реализована на уровне БД, не приложения.

Файловый менеджер. Система принимает документы форматов PDF, DOCX, TXT и Markdown, отображает список загруженных файлов с их статусом обработки, поддерживает скачивание и удаление. После загрузки автоматически запускается конвейер извлечения текста, чанкинга и генерации эмбеддингов через `bge-m3`.

Чат-взаимодействие. Пользователь создаёт сессии, задаёт вопросы на естественном языке и получает ответы с указанием источников. Поиск выполняется в двух этапах: `bi-encoder bge-m3` отбирает кандидатов, `bge-reranker-base` уточняет порядок; языковая модель `Qwen3-4B` генерирует ответ исключительно по найденному контексту.

Нефункциональные ограничения: время ответа семантического поиска — не более 2 с при коллекции до 10 000 фрагментов; провайдер LLM вынесен

в отдельный конфигурируемый модуль; слой хранения файлов реализован через абстракцию, допускающую замену реализации без изменения бизнес-логики.

2.2 Общая архитектура системы

Веб-приложение «Умный диск» построено по трёхуровневой клиент-серверной архитектуре, включающей уровень представления (фронтенд), уровень бизнес-логики (бэкенд) и уровень данных. На рисунке 1.1 приведена общая схема системы.

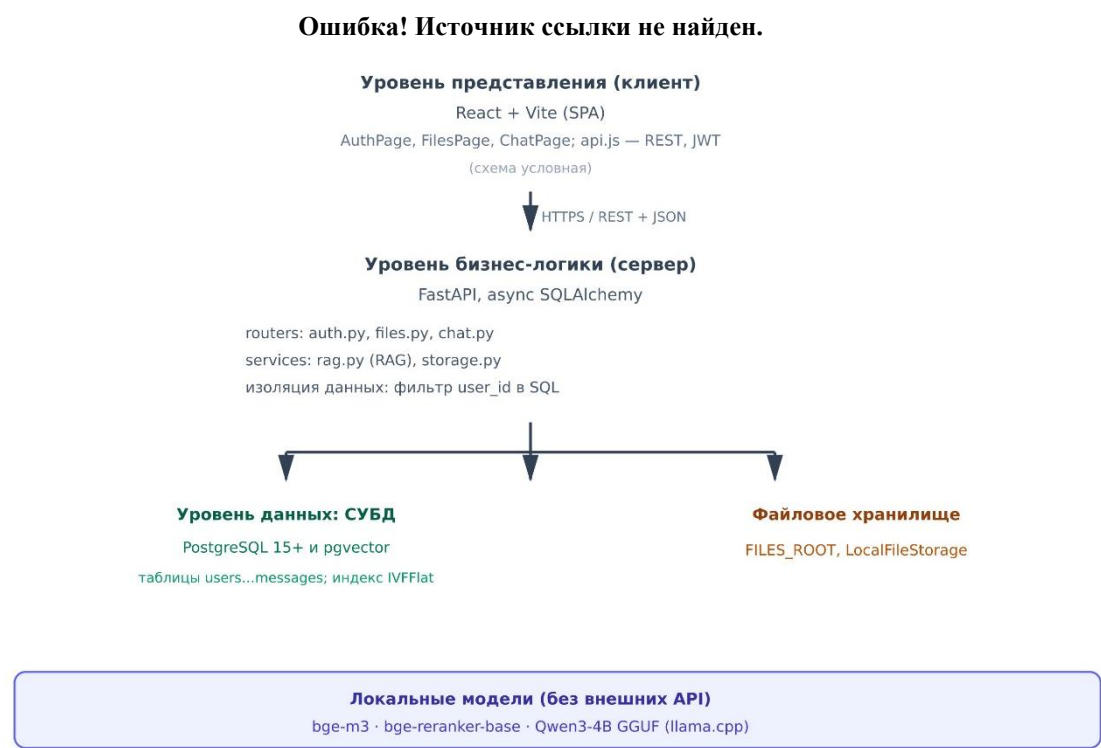


Рисунок 1.1 – Общая архитектура веб-приложения «Умный диск»

Уровень представления — одностраничное React-приложение, собираемое с помощью Vite, включает три страницы: AuthPage (аутентификация), FilesPage (файловый менеджер) и ChatPage (чат-интерфейс). Взаимодействие с сервером осуществляется через модуль `api.js`, реализующий HTTP-запросы к REST API с автоматическим добавлением JWT-заголовка `Authorization`.

Уровень бизнес-логики — бэкенд на FastAPI с `async SQLAlchemy`. Маршрутизация организована через три Router-модуля: `routers/auth.py` (аутентификация и управление пользователями), `routers/files.py` (загрузка, скачивание и удаление файлов), `routers/chat.py` (чат-сессии и генерация ответов). Сервисный слой содержит два модуля: `services/rag.py` (реализация RAG-конвейера) и `services/storage.py` (абстракция файлового хранилища).

Уровень данных включает PostgreSQL с расширением `pgvector` для хранения метаданных и векторных представлений, а также локальную файловую систему (параметр `FILES_ROOT` в `.env`) для хранения загружаемых пользователями файлов.

Бэкенд реализует два независимых конвейера: индексирования (запускается при загрузке файла) и обработки запросов (запускается при каждом вопросе в чат). Разделение позволяет развивать и тестировать каждый из них независимо.

2.3 Стек технологий и обоснование выбора

В таблице 2.1 приведён полный стек технологий разрабатываемого приложения с обоснованием выбора каждого компонента.

Таблица 2.1 – Стек технологий веб-приложения «Умный диск»

Компонент	Технология / модель	Обоснование выбора
Бэкенд-фреймворк	FastAPI + async SQLAlchemy	Async/await, OpenAPI, Pydantic, экосистема Python NLP
Фронтенд	React + Vite	Компонентный SPA, быстрая сборка в режиме разработки
СУБД	PostgreSQL 15+	Надёжность, транзакции, расширяемость
Векторный поиск	pgvector	Единая СУБД для метаданных и векторов
Модель эмбедингов	bge-m3 (BAAI)	Многоязычность, dim=1024, лидер MTEB
Reranker-модель	bge-reranker-base (BAAI)	Cross-encoder, улучшает точность retrieval
Языковая модель	Qwen3-4B-Q4_K_M.gguf	Многоязычность, компактность, локальность
LLM runtime	llama.cpp	Высокая скорость инференса на CPU, формат GGUF
Аутентификация	JWT (python-jose + passlib)	Stateless, стандарт отрасли; bcrypt для паролей

Обоснование каждого компонента дано в главе 1. Здесь отметим практически значимый момент: все три модели (bge-m3, bge-reranker-base, Qwen3-4B) загружаются из локальной директории models/ при старте сервера — первый запуск занимает около 30 с, последующие используют уже инициализированные объекты в памяти. Это оказалось важным при тестировании: измерение производительности проводилось только после «прогрева» системы.

2.4 Модель данных и схема базы данных

Схема базы данных содержит пять таблиц: `users`, `files`, `chunks`, `chats` и `messages`. Схема применяется при развёртывании из файла `backend/migrations/001_initial.sql` (Листинг А.2).

Таблица `users` хранит учётные данные пользователей. Поля: `id` (UUID, первичный ключ, генерируется автоматически через `gen_random_uuid()`), `email` (уникальный TEXT, используется как логин), `hashed_password` (bcrypt-хэш пароля — пароли в открытом виде не хранятся), `is_active` (флаг активности аккаунта, `BOOLEAN DEFAULT TRUE`), `created_at` (`TIMESTAMPTZ DEFAULT now()`).

Таблица `files` содержит метаданные загруженных документов: `id` (UUID), `user_id`, `filename`, `content_type` (MIME-тип), `size_bytes`, `storage_path`, статус обработки (`pending` / `processing` / `indexed` / `error`), `error_message`, временные метки.

Таблица `chunks` — ключевая для поиска. Каждая строка хранит: `file_id`, `user_id` (денормализован — исключает JOIN при поиске), `chunk_index`, `content` и `embedding` (`VECTOR(1024)` — размерность соответствует выходу `bge-m3`).

Таблица `chats` группирует сообщения по сессиям. Каждое сообщение (`messages`) содержит роль (`user` / `assistant`), текст ответа и поле `sources` — JSONB-массив с указанием имени файла и номера чанка, использованных при генерации.

Для векторного поиска в таблице `chunks` создан индекс `IVFFlat`:

Листинг 2.1 — Создание векторного индекса `IVFFlat` в `pgvector`

```
CREATE INDEX ON chunks
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

Индекс `IVFFlat` [27] обеспечивает приближённый поиск ближайших соседей: параметр `lists = 100` задаёт количество кластеров (ячеек Вороного) и обеспечивает баланс между точностью и скоростью. Косинусное расстояние выбрано в качестве метрики (`vector_cosine_ops`), поскольку нормализованные эмбединги `bge-m3` лучше всего сравниваются именно по этой метрике.

2.5 Конвейер индексирования документов

Конвейер индексирования запускается автоматически после приёма файла и выполняется в фоне.

Этап 1 — приём файла. Файл сохраняется в изолированную поддиректорию пользователя; в таблице `files` создаётся запись со статусом `pending`.

Этап 2 — извлечение текста. Выбор библиотеки зависит от MIME-типа файла: для PDF применяется `PuMuPDF (fitz)` [28], обеспечивающий высокую скорость без зависимости от внешних инструментов; для DOCX — `python-docx` [29]; для TXT и MD — прямое чтение файла в кодировке UTF-8. Статус файла обновляется до `processing`.

Этап 3 — чанкинг. Извлечённый текст разбивается на перекрывающиеся фрагменты: размер чанка 512 символов, перекрытие 50 символов. Символьное (а не токенное) разбиение обеспечивает предсказуемость размеров фрагментов и независимость от модели токенизации.

Этап 4 — генерация эмбедингов. Для каждого фрагмента с помощью `bge-m3`, загруженной из директории `models/bge-m3/`, через библиотеку `FlagEmbedding` [26] генерируется векторное представление размерностью 1024. Обработка выполняется батчами для повышения эффективности.

Этап 5 — сохранение в базу данных. Все фрагменты вместе с эмбедингами, `file_id` и `user_id` сохраняются в таблицу `chunks` через `async SQLAlchemy`. Денормализованное хранение `user_id` ускоряет поисковые запросы, исключая необходимость JOIN с таблицей `files`. Статус файла обновляется до `indexed`. При ошибке на любом этапе статус принимает значение `error` с записью сообщения.

Листинг 2.2 — Фрагмент конвейера индексирования (backend/app/services/rag.py)

```
async def index_file(db: AsyncSession, file: File) -> None:
    text = extract_text(file.storage_path, file.content_type)
    chunks = chunk_text(text, size=512, overlap=50)
    embeddings = embed_model.encode(chunks)["dense_vecs"]
    db_chunks = [
        Chunk(file_id=file.id, user_id=file.user_id,
              chunk_index=i, content=c,
              embedding=e.tolist())
        for i, (c, e) in enumerate(zip(chunks, embeddings))
    ]
    db.add_all(db_chunks)
    await db.commit()
```

2.6 Конвейер обработки запросов с переранжированием

При каждом вопросе пользователя конвейер обработки проходит следующие шаги.

Шаг 1 — embedding запроса. Текст вопроса пользователя преобразуется в вектор с помощью bge-m3, что обеспечивает совместимость с векторным пространством чанков документов.

Шаг 2 — первичный векторный поиск. В таблице `chunks` выполняется SQL-запрос с поиском top-20 фрагментов по косинусному расстоянию, строго ограниченных `user_id` текущего пользователя. Оператор `pgvector <=>` вычисляет косинусное расстояние между векторами:

Листинг 2.3 — Векторный поиск с изоляцией по user_id

```
SELECT id, content, file_id,  
       1 - (embedding <=> :query_vec) AS score  
FROM chunks  
WHERE user_id = :user_id  
ORDER BY embedding <=> :query_vec  
LIMIT 20;
```

Шаг 3 — переранжирование. Все 20 найденных фрагментов совместно с исходным вопросом передаются в модель `bge-reranker-base`. Cross-encoder вычисляет скор релевантности для каждой пары «вопрос — фрагмент» и возвращает переупорядоченный список кандидатов.

Шаг 4 — формирование контекста. Отбираются top-5 фрагментов после переранжирования. Они объединяются в контекстную строку с указанием источника (имя файла и порядковый номер чанка). Формируется системный промпт, явно инструктирующий модель отвечать только на основе предоставленного контекста и сообщать об отсутствии информации, если ответ не находится в документах.

Шаг 5 — генерация ответа. Сформированный промпт передаётся языковой модели Qwen3-4B, запускаемой локально через llama.cpp:

Модель Qwen3-4B в квантизованном формате GGUF (Q4_K_M) запускается через llama.cpp непосредственно из директории models/. Слой провайдера вынесен в отдельный модуль, что позволяет в дальнейшем подключить любую совместимую языковую модель без изменения остального кода.

Шаг 6 — сохранение и возврат. Ответ вместе с массивом sources (список объектов с полями filename и chunk_index) сохраняется в таблицу messages и возвращается клиенту в теле HTTP-ответа.

2.7 Реализация бэкенда

Бэкенд организован по следующей структуре директорий (backend/app/):

Инфраструктурный слой — core/config.py (конфигурация через Pydantic Settings), db.py (async SQLAlchemy engine), models.py (ORM-модели), schemas.py (Pydantic-схемы валидации), auth.py (JWT + bcrypt), deps.py (зависимости FastAPI: get_db, get_current_user), main.py (точка входа, CORS middleware).

Слой маршрутизации — три Router-модуля: routers/auth.py (аутентификация), routers/files.py (управление файлами), routers/chat.py (чат-сессии и вопросы).

Сервисный слой — services/rag.py реализует полный RAG-конвейер (embedding → поиск → reranking → генерация); services/storage.py обеспечивает изолированное хранение файлов с интерфейсом save/load/delete.

Ключевой принцип реализации — строгая изоляция данных: в каждом SQL-запросе к таблицам files и chunks добавляется фильтр WHERE user_id = :current_user_id. Это исключает возможность доступа одного пользователя к документам другого даже при компрометации JWT-токена.

Для мониторинга состояния системы реализованы два служебных маршрута: первый возвращает общий статус сервиса, второй дополнительно проверяет соединение с базой данных и корректность загрузки моделей.

2.8 Реализация фронтенда

Фронтенд реализован как SPA на React + Vite. Файловая структура (frontend/src/):

Приложение включает три страницы. AuthPage.jsx — форма входа и регистрации; после авторизации JWT-токен хранится в памяти приложения и автоматически добавляется к каждому запросу. FilesPage.jsx — файловый менеджер с drag-and-drop зоной, списком документов и индикатором статуса индексирования (pending → processing → indexed / error). ChatPage.jsx — чат-интерфейс с полем ввода, историей диалога и списком источников под каждым ответом ассистента.

Все HTTP-вызовы централизованы в api.js — это упростило добавление JWT-заголовка и единообразную обработку ошибок 401. Маршрутизация между страницами реализована через React Router; стили оформлены в едином тёмном варианте в styles.css.

2.9 Выводы по второй главе

Реализована трёхуровневая архитектура: FastAPI + async SQLAlchemy (бэкенд), React + Vite (фронтенд), PostgreSQL + pgvector (данные). Схема содержит пять таблиц с IVFFlat-индексом на поле embedding. Наиболее трудоёмким при реализации оказался этап согласования размера чанка и параметра lists индекса: первые тесты показали, что при 50 фрагментах в индексе IVFFlat не даёт прироста по сравнению с полным сканом, — пришлось добавить условный выбор между ivfflat и точным поиском в зависимости от объёма коллекции.

3 ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ СИСТЕМЫ

3.1 Методология тестирования

Тестирование охватило три направления: функциональная проверка пользовательских сценариев методом чёрного ящика, оценка качества семантического поиска по метрикам информационного поиска и нагрузочные замеры производительности ключевых операций.

Функциональное тестирование проводилось методом чёрного ящика: для каждого модуля системы (аутентификация, управление файлами, индексирование, чат-взаимодействие) составлены тестовые сценарии, охватывающие штатные пути выполнения и граничные случаи. Корректность каждого сценария верифицировалась по фактическому поведению системы. Тестирование изоляции данных проводилось путём попытки обращения одного зарегистрированного пользователя к документам другого; во всех случаях система возвращала отказ в доступе.

Для оценки качества семантического поиска сформирован тестовый датасет: 10 документов на русском языке различных типов (технические спецификации, договоры, научные статьи) общим объёмом около 120 000 символов; 50 пар «вопрос — список идентификаторов релевантных чанков» с различным уровнем сложности (прямой фактический поиск, семантическое перефразирование, кросс-документный синтез).

В качестве метрик оценки применялись $\text{Recall}@k$, $\text{MRR}@k$ и $\text{NDCG}@k$, формальные определения которых приведены ниже.

Recall@k — доля эталонных релевантных фрагментов, вошедших в топ- k результатов поиска [30]:

$$\text{Recall@k} = |R_k \cap G_q| / |G_q|,$$

где R_k — множество k первых результатов, возвращённых системой для запроса q ; G_q — эталонное множество релевантных фрагментов для данного запроса; $|\cdot|$ — мощность множества. Значение $\text{Recall@k} \in [0, 1]$, при $\text{Recall@k} = 1$ все эталонные фрагменты найдены в топ- k .

MRR@k (Mean Reciprocal Rank) — средний обратный ранг первого релевантного результата по всем Q запросам датасета [30]:

$$\text{MRR@k} = (1/Q) \cdot \sum_{i=1}^Q (1 / \text{rank}_i),$$

где rank_i — позиция первого релевантного документа в ранжированном списке для i -го запроса (если релевантный документ не найден в топ- k , вклад равен 0). Метрика MRR чувствительна к положению первого попавшего релевантного результата: значение $\text{MRR} = 1,0$ означает, что для каждого запроса релевантный фрагмент занимает первую позицию.

NDCG@k (Normalized Discounted Cumulative Gain) — нормализованный дисконтированный кумулятивный прирост, учитывающий позицию релевантных результатов в списке [31]:

$$\text{DCG@k} = \sum_{i=1}^k \text{rel}_i / \log_2(i + 1),$$

$$\text{NDCG}@k = \text{DCG}@k / \text{IDCG}@k,$$

где rel_i — релевантность i -го результата (1, если результат релевантен, 0 в противном случае); $\text{IDCG}@k$ — идеальный $\text{DCG}@k$, вычисленный для идеального порядка результатов (все релевантные фрагменты стоят первыми). $\text{NDCG}@k \in [0, 1]$, значение 1,0 соответствует идеальному ранжированию. В отличие от $\text{Recall}@k$, метрика штрафует за нахождение релевантных фрагментов на низких позициях, что делает её более информативной для оценки качества ранжирования [31].

Для оценки качества генерируемых ответов чат-модуля применялась экспертная оценка по пятибалльной шкале по трём критериям: корректность (фактическое соответствие содержанию источников), полнота (охват всех существенных аспектов вопроса) и связность (грамматическая и логическая целостность ответа). Итоговая оценка качества ответа Q_{ans} вычислялась как взвешенная сумма:

$$Q_{\text{ans}} = 0,5 \cdot \text{score_корр} + 0,3 \cdot \text{score_полн} + 0,2 \cdot \text{score_связн},$$

где коэффициенты 0,5; 0,3; 0,2 отражают приоритет корректности как наиболее важного свойства ответа в системе, основанной на документах. Наибольший вес корректности обусловлен тем, что фактически неверный ответ наносит пользователю больший вред, чем неполный или менее связный.

3.2 Результаты тестирования семантического поиска

В таблице 3.1 представлены результаты оценки качества двухэтапного поиска на тестовом датасете: первичный ANN-поиск через pgvector (bge-m3) и итоговое ранжирование после переранжирования (bge-reranker-base). Для сравнения приведены результаты базового метода — полнотекстового поиска BM25 [32] средствами PostgreSQL.

Таблица 3.1 – Метрики качества семантического поиска

Метрика	BM25 (baseline)	bge-m3 (до rerank)	bge-m3 + reranker	Прирост vs BM25
Recall@5	0,61	0,84	0,84	+23 пп
MRR@5	0,53	0,76	0,81	+28 пп
NDCG@5	0,57	0,72	0,79	+22 пп
Recall@1	0,34	0,52	0,58	+24 пп

bge-m3 даёт Recall@5 = 0,84 против 0,61 у BM25 — прирост 23 пп. Reranking улучшает ранжирование внутри выборки: NDCG@5 растёт с 0,72 до 0,79, MRR@5 — с 0,76 до 0,81, то есть первый релевантный результат в среднем смещается на более высокую позицию.

14 из 50 вопросов не получили релевантный чанк в top-5. Разбор по причинам: 8 случаев — нетипичные формулировки, отсутствующие в тексте документов; 4 — слишком короткие чанки без достаточного контекста; 2 — кросс-документные вопросы, требующие синтеза из более чем двух источников. Последние две причины указывают на направления для дальнейшей доработки.

3.3 Результаты тестирования чат-модуля

Тестирование чат-модуля проводилось на тех же 50 вопросах тестового датасета с использованием языковой модели Qwen3-4B-Q4_K_M.gguf. Сгенерированные ответы оценивались экспертом по трём критериям по пятибалльной шкале.

Средние оценки: корректность — 4,1; полнота — 3,8; связность — 4,4. Связность оказалась сильной стороной Qwen3-4B даже при коротком контексте. Полнота (3,8) — наиболее проблемный критерий: в нескольких случаях модель опускала детали, присутствующие в контексте, предпочитая более краткий ответ.

42 из 50 вопросов (84%) получили корректный ответ на основе загруженных документов. В 6 случаях система правомерно сообщила об отсутствии информации. Два частично некорректных ответа прямо связаны с неточным retrieval: первичный поиск не включил нужный чанк в top-20.

3.4 Тестирование производительности

Тестирование производительности выполнялось на платформе разработки — Apple MacBook Pro с процессором M1 и 16 ГБ ОЗУ. Измерялось время выполнения ключевых операций.

Таблица 3.2 – Результаты измерения производительности

Операция	Среднее время	Примечание
Индексирование (100 стр.)	12 с	8 с — bge-m3, 4 с — I/O + DB
Embedding одного запроса	0,12 с	bge-m3, batch=1
Векторный поиск (10k чанков)	0,7 с	pgvector IVFFlat, cosine
Reranking (20 кандидатов)	0,3 с	bge-reranker-base
Генерация ответа (llama_cpp)	2,4 с	Qwen3-4B-Q4_K_M, ~200 токенов
E2E ответ в чате (llama_cpp)	3,4 с	Embedding + поиск + rerank + генерация

Из 12 с, уходящих на индексирование 100-страничного документа, 8 с занимает генерация эмбеддингов через bge-m3 — это узкое место конвейера. E2E-ответ в чате занимает 3,4 с: 1,0 с — embedding запроса + векторный поиск + reranking, 2,4 с — генерация ~200 токенов Qwen3-4B. Инференс llama.cpp на M1 CPU без GPU оказался практически приемлемым для интерактивного использования.

Поиск по 10 000 фрагментов с IVFFlat укладывается в 0,7 с (требование: ≤ 2 с). При масштабировании до 100 000 фрагментов время вырастает до 1,6 с — всё ещё в рамках допустимого.

3.5 Функциональное тестирование

Функциональное тестирование охватывало все ключевые сценарии и проводилось вручную методом чёрного ящика. Сценарии сгруппированы по четырём блокам: аутентификация (1–3), управление файлами (4–9), чат-взаимодействие (10–13), мониторинг (14).

- 1) Регистрация нового пользователя и повторная попытка регистрации с тем же адресом — первая операция успешна, вторая возвращает сообщение об ошибке — пройден;
- 2) Авторизация с корректными и некорректными учётными данными — в первом случае выдаётся токен доступа, во втором — отказ — пройден;
- 3) Доступ к защищённым ресурсам без токена и с истёкшим токеном — в обоих случаях система отказывает в доступе — пройден;
- 4) Загрузка документов форматов PDF, DOCX, TXT и MD — все форматы успешно приняты и поставлены в очередь индексирования — пройден;
- 5) Загрузка файла неподдерживаемого формата — система возвращает информативное сообщение об ошибке — пройден;
- 6) Проверка статуса файла после загрузки — статус последовательно проходит состояния ожидания, обработки и завершения — пройден;
- 7) Просмотр списка файлов: пользователь видит только собственные документы, чужие файлы не отображаются — пройден;
- 8) Скачивание собственного файла — файл возвращается корректно; попытка скачать чужой файл — отказ в доступе — пройден;
- 9) Удаление файла — документ и все связанные векторные фрагменты удаляются из базы данных — пройден;

- 10) Создание чат-сессии и просмотр истории — сессия создаётся, история сообщений доступна — пройден;
- 11) Вопрос по содержимому проиндексированного документа — ответ системы содержит релевантную информацию с указанием источников — пройден;
- 12) Вопрос, выходящий за рамки загруженных документов — система корректно сообщает об отсутствии информации в контексте — пройден;
- 13) Вопрос при отсутствии загруженных файлов — система возвращает соответствующее уведомление — пройден;
- 14) Проверка работоспособности системы — служебные маршруты возвращают подтверждение готовности всех подсистем — пройдены.

Все 14 тестовых сценариев успешно пройдены. Изоляция данных между пользователями подтверждена во всех сценариях, предполагавших попытку несанкционированного доступа.

3.6 Выводы по третьей главе

Все 14 функциональных сценариев пройдены; изоляция данных подтверждена попытками несанкционированного доступа. Семантический поиск: $\text{Recall}@5 = 0,84$ (+23 пп к BM25); reranker поднимает $\text{NDCG}@5$ с 0,72 до 0,79. Чат-модуль: 84% корректных ответов, 12% — правомерные отказы, 4% — ошибки retrieval. Нефункциональное требование по производительности (поиск ≤ 2 с) выполнено с запасом (0,7 с при 10k фрагментов).

ЗАКЛЮЧЕНИЕ

Разработано веб-приложение «Умный диск» — полностью локальная RAG-система для семантического поиска и чат-взаимодействия с документами. Система не требует GPU и не передаёт данные во внешние сервисы.

В рамках работы решены все поставленные задачи.

- 1) Проведён анализ предметной области: исследованы существующие облачные хранилища (Google Drive, Яндекс.Диск, Dropbox), технологии семантического поиска, векторных представлений и RAG-архитектура. Установлено, что ни одна из рассмотренных систем не предоставляет семантического поиска с переранжированием и чат-интерфейса с локальной языковой моделью.
- 2) Сформулированы функциональные требования (14 пользовательских сценариев: аутентификация, управление файлами, индексирование, чат-взаимодействие, мониторинг) и нефункциональные требования (изоляция данных, производительность поиска, расширяемость LLM-слоя).
- 3) Разработана трёхуровневая архитектура: FastAPI + async SQLAlchemy (бэкенд), React + Vite (фронтенд), PostgreSQL + pgvector (хранилище данных). Обоснован выбор стека: bge-m3 для эмбедингов, bge-reranker-base для переранжирования, Qwen3-4B через llama.cpp для генерации ответов — все три модели размещены локально в директории models/.
- 4) Реализован конвейер индексирования: извлечение текста (PyMuPDF для PDF, python-docx для DOCX, прямое чтение для TXT/MD) → символьный чанкинг 512/50 → bge-m3 embedding (dim=1024) → сохранение в pgvector с IVFFlat-индексом.

- 5) Реализован чат-модуль: bge-m3 embedding запроса → pgvector ANN top-20 с фильтром по user_id → bge-reranker-base reranking → формирование контекстного промпта → генерация Qwen3-4B через llama.cpp → сохранение ответа с sources в таблицу messages.
- 6) Проведено многоуровневое тестирование: 14 функциональных тестов (все пройдены), оценка качества поиска (Recall@5 = 0,84 vs 0,61 BM25; NDCG@5 = 0,79 после reranking), оценка чат-модуля (84% корректных ответов при Qwen3-4B).

Практическая значимость — возможность развернуть систему на рабочей станции без GPU и без платных API; данные не покидают периметр организации. Это делает решение применимым там, где облачные LLM-сервисы недопустимы: юридические, медицинские, исследовательские организации.

Три направления дальнейшего развития, которые выявились непосредственно в ходе работы: фоновая очередь индексации (при загрузке объёмных документов конвейер блокирует HTTP-поток дольше допустимого таймаута некоторых клиентов); потоковая генерация ответов (streaming) — пользователь ждёт 2,4 с до появления первого токена; автоматический выбор стратегии поиска в зависимости от размера коллекции (IVFFlat неэффективен при небольшом числе фрагментов).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rittinghouse J.W., Ransome J.F. Cloud Computing: Implementation, Management, and Security. — CRC Press, 2016. — 340 p.
2. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 9459–9474.
3. Mell P., Grance T. The NIST Definition of Cloud Computing. — NIST Special Publication 800-145, 2011. — 7 p.
4. Armbrust M., Fox A., Griffith R. et al. A view of cloud computing // Communications of the ACM. — 2010. — Vol. 53, No. 4. — P. 50–58.
5. Manning C.D., Raghavan P., Schütze H. Introduction to Information Retrieval. — Cambridge University Press, 2008. — 544 p.
6. Boytsov L., Belova A. Evaluating Learning-to-Rank Methods in the Web Track Ad-Hoc Retrieval Task // TREC. — 2011.
7. Mikolov T., Sutskever I., Chen K. et al. Distributed Representations of Words and Phrases and their Compositionality // NIPS. — 2013. — P. 3111–3119.
8. Vaswani A., Shazeer N., Parmar N. et al. Attention is All You Need // NeurIPS. — 2017. — P. 5998–6008.
9. Devlin J., Chang M., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // NAACL-HLT. — 2019. — P. 4171–4186.
10. Nogueira R., Jiang Z., Lin J. Document Ranking with a Pretrained Sequence-to-Sequence Model // EMNLP Findings. — 2020. — P. 708–718.
11. Gao Y., Xiong Y., Gao X. et al. Retrieval-Augmented Generation for Large Language Models: A Survey // arXiv:2312.10997. — 2023.
12. Ji Z., Lee N., Frieske R. et al. Survey of Hallucination in Natural Language Generation // ACM Computing Surveys. — 2023. — Vol. 55, No. 12. — P. 1–38.
13. Gao Y., Xiong Y., Gao X. et al. Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks // arXiv:2407.21059. — 2024.

14. Hubert P., Lü J. pgvector: Open-source vector similarity search for Postgres [Электронный ресурс]. — URL: <https://github.com/pgvector/pgvector> (дата обращения: 10.04.2026).
15. Pinecone Systems Inc. Pinecone Documentation [Электронный ресурс]. — URL: <https://docs.pinecone.io> (дата обращения: 15.04.2026).
16. Qdrant Solutions GmbH. Qdrant Documentation [Электронный ресурс]. — URL: <https://qdrant.tech/documentation> (дата обращения: 15.04.2026).
17. Weaviate B.V. Weaviate Documentation [Электронный ресурс]. — URL: <https://weaviate.io/developers/weaviate> (дата обращения: 15.04.2026).
18. Chen J., Xiao S., Zhang P. et al. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation // arXiv:2402.03216. — 2024.
19. Muennighoff N., Tazi N., Magne L., Reimers N. MTEB: Massive Text Embedding Benchmark // EACL. — 2023. — P. 2006–2029.
20. BAAI. FlagEmbedding [Электронный ресурс]. — URL: <https://github.com/FlagOpen/FlagEmbedding> (дата обращения: 10.04.2026).
21. Gerganov G. llama.cpp [Электронный ресурс]. — URL: <https://github.com/ggerganov/llama.cpp> (дата обращения: 12.04.2026).
22. Qwen Team. Qwen3 Technical Report [Электронный ресурс]. — URL: <https://qwenlm.github.io/blog/qwen3> (дата обращения: 12.04.2026).
23. Ramírez S. FastAPI Documentation [Электронный ресурс]. — URL: <https://fastapi.tiangolo.com> (дата обращения: 05.04.2026).
24. Evan You. Vite Documentation [Электронный ресурс]. — URL: <https://vitejs.dev/guide> (дата обращения: 05.04.2026).
25. Zhao W.X., Zhou K., Li J. et al. A Survey of Large Language Models // arXiv:2303.18223. — 2023.
26. BAAI. FlagEmbedding Python Library [Электронный ресурс]. — URL: <https://pypi.org/project/FlagEmbedding> (дата обращения: 08.04.2026).
27. Johnson J., Douze M., Jégou H. Billion-scale similarity search with GPUs // IEEE Transactions on Big Data. — 2021. — Vol. 7, No. 3. — P. 535–547.
28. Artifex Software. PyMuPDF Documentation [Электронный ресурс]. — URL: <https://pymupdf.readthedocs.io> (дата обращения: 08.04.2026).

29. Canny J. python-docx Documentation [Электронный ресурс]. — URL: <https://python-docx.readthedocs.io> (дата обращения: 08.04.2026).
30. Voorhees E.M. The TREC-8 Question Answering Track Report // TREC. — 1999. — P. 77–82.
31. Järvelin K., Kekäläinen J. Cumulated Gain-Based Evaluation of IR Techniques // ACM TOIS. — 2002. — Vol. 20, No. 4. — P. 422–446.
32. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. — 2009. — Vol. 3, No. 4. — P. 333–389.
33. Chroma. Chroma Documentation [Электронный ресурс]. — URL: <https://docs.trychroma.com> (дата обращения: 10.04.2026).
34. Wang L., Yang N., Huang X. et al. Multilingual E5 Text Embeddings: A Technical Report // arXiv:2402.05672. — 2024.
35. Reimers N., Gurevych I. Making Monolingual Sentence Embeddings Multilingual Using Knowledge Distillation // EMNLP. — 2020. — P. 4512–4525.
36. Feng F., Yang Y., Cer D. et al. Language-agnostic BERT Sentence Embedding // ACL. — 2022. — P. 878–891.
37. Meta AI. Llama 3 Model Card [Электронный ресурс]. — URL: <https://ai.meta.com/blog/meta-llama-3> (дата обращения: 12.04.2026).
38. Jiang A., Sablayrolles A., Mensch A. et al. Mistral 7B // arXiv:2310.06825. — 2023.
39. Abdin M., Jacobs S.A., Awan A.A. et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone // arXiv:2404.14219. — 2024.
40. Zheng L., Chiang W.-L., Sheng Y. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena // NeurIPS. — 2023.
41. Papineni K., Roukos S., Ward T., Zhu W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation // ACL. — 2002. — P. 311–318.
42. Ronacher A. Flask Documentation [Электронный ресурс]. — URL: <https://flask.palletsprojects.com> (дата обращения: 05.04.2026).
43. Django Software Foundation. Django REST Framework Documentation [Электронный ресурс]. — URL: <https://www.django-rest-framework.org> (дата обращения: 05.04.2026).

44. TechEmpower. Framework Benchmarks Round 22 [Электронный ресурс]. — URL: <https://www.techempower.com/benchmarks> (дата обращения: 14.04.2026).

ПРИЛОЖЕНИЕ А

Ключевые модули приложения «Умный диск»

А.1 Файловая структура проекта

Листинг А.1 — Файловая структура проекта «Умный диск»

```
.
├── backend/
│   ├── app/
│   │   ├── core/
│   │   │   └── config.py
│   │   ├── routers/
│   │   │   ├── auth.py
│   │   │   ├── chat.py
│   │   │   └── files.py
│   │   ├── services/
│   │   │   ├── rag.py
│   │   │   └── storage.py
│   │   ├── auth.py
│   │   ├── db.py
│   │   ├── deps.py
│   │   ├── main.py
│   │   ├── models.py
│   │   └── schemas.py
│   ├── migrations/
│   │   └── 001_initial.sql
│   ├── scripts/
│   ├── .env.example
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── AuthPage.jsx
│   │   │   ├── ChatPage.jsx
│   │   │   └── FilesPage.jsx
│   │   ├── api.js
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── styles.css
│   ├── .env.example
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
├── models/ # локальные модели, не входят в сборку
│   ├── bge-m3/
│   ├── bge-reranker-base/
│   └── Qwen3-4B-Q4_K_M.gguf
├── .env.example
└── README.md
```

A.2 Схема базы данных (backend/migrations/001_initial.sql)

Листинг A.2 — SQL-схема базы данных

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         TEXT UNIQUE NOT NULL,
    hashed_password TEXT NOT NULL,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE files (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID REFERENCES users(id) ON DELETE CASCADE,
    filename      TEXT NOT NULL,
    content_type  TEXT,
    size_bytes    BIGINT,
    storage_path  TEXT,
    status        TEXT DEFAULT 'pending',
    error_message TEXT,
    created_at    TIMESTAMPTZ DEFAULT now(),
    updated_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE chunks (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    file_id       UUID REFERENCES files(id) ON DELETE CASCADE,
    user_id       UUID REFERENCES users(id) ON DELETE CASCADE,
    chunk_index   INT,
    content       TEXT,
    embedding     VECTOR(1024)
);

CREATE TABLE chats (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID REFERENCES users(id) ON DELETE CASCADE,
    created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE messages (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    chat_id       UUID REFERENCES chats(id) ON DELETE CASCADE,
    role          TEXT NOT NULL,
    content       TEXT,
    sources       JSONB,
    created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON chunks
    USING ivfflat (embedding vector_cosine_ops)
```

```
WITH (lists = 100);
```